

Awarding Body: Department of Computing

Programme Name: Msc Data Science

Module Name (and Part if applicable): Programming for Data Analytics

Assessment Title: Project: Sales Data Preparation and Analysis using Python

Student Number: 25265702

Tutor Name: Muhammad Saqib

Word Count: 3227

Please refer to the Word Count Policy on your Module Page for guidance

LIST OF CONTENTS

1. INTRODUCTION	Page 1
2. TOOLS AND TECHNOLOGIES	Pages 2 – 4
3. DATASET DESCRIPTION	Page 5
4. ANALYSIS OF THE SALES DATASET	Pages 6 – 19
4.1 Data Understanding	Pages 6 – 9
4.2 Data Preparation	Pages 9 – 12
4.2.1 Data Cleaning	Page 9
4.2.2 Handling Missing Data	Page 10
4.2.3 Data Transformation	Page 10
4.2.4 Outlier Detection	Pages 10,11
4.2.5 Data Reduction and Feature Preparation ---	Page 12
4.3 Data Analysis	Pages 13 – 17
4.3.1 Descriptive Analysis	Page 13
4.3.2 Salesperson Performance Analysis	Pages 13,14
4.3.3 Priority Based Sales Analysis	Pages 14,15
4.3.4 Shipping Mode Analysis	Pages 15,16
4.3.5 Time Based Analysis	Pages 16,17
4.4 Data Visualisation	Pages 17 – 19
4.4.1 Histogram	Page 17
4.4.2 Bar Charts	Pages 17,18
4.4.3 Box Plots	Page 18
4.4.4 Scatter Diagram	Page 18
4.4.5 Pie Charts	Page 19
4.4.6 Line Diagrams	Page 19
4.4.7 Correlation Heatmap Analysis	Page 19
5. CONCLUSION	Pages 20 – 22
5.1 Results and Discussions	Pages 20,21
5.2 Reflective Report	Pages 21,22
6. REFERENCES	Page 23
7. APPENDIX	Pages 24 - 46

PROJECT: SALES DATA PREPARATION AND ANALYSIS USING PYTHON FOR DATA ANALYTICS

1. INTRODUCTION

Nowadays the process of decision making in organisations are highly dependant on data analytics. It helps in improving operational performance and in identifying business trends. Data analytics involves the collection, processing and interpretation of data to generate meaningful insights that can assist organisations in strategic planning and problem solving (Arden University, 2024). Since digital technologies are expanding and online systems are getting more composite, organisations are now creating large amounts of data every day. Hence it is very much essential to have proper analytical tools and methodologies to tackle this data.

Python is one of the most commonly used programming languages for data analytics because of its simplicity, flexibility and extensive availability of analytical libraries. Libraries such as Pandas, NumPy and Matplotlib enable efficient data manipulation, numerical computation and visulisation, making Python highly suitable for data science applications (Joshi and Tiwari, 2023). Recent studies have also highlighted Python's importance in modern data analytics due to its ability to streamline data preparation, cleaning, transformation and visualisation processes (Kabir et al., 2024)

In this project exploratory and analytical procedures are performed in the sales dataset produced by the company One Time Sales and Marketing Ltd. This dataset shows sales related information such as customer details, shipping method, sales personnel, order priorities, sales value etc. But raw dataset may contain inconsistencies, duplicates, missing values or other issues in formatting which may affect the process of data analysis and the results generated. So, data preparation is considered one of the most critical stages within the data analytics lifecycle (Arden University, 2024). Proper data cleaning and transformation helps in making sure that the dataset is accurate, structured and is suitable for further analytical procedures.

This project report shows the importance of data preparation, analysis and visualisation in creating insights which are meaningful for the company and shows how Python is used for the same on the sales dataset.

2. TOOLS AND TECHNOLOGIES

Python was chosen as the most appropriate programming language for this project because of its simplicity, flexibility and extensive support for data analytics applications. Python is widely recognised as one of the leading programming languages in data science due to its ability to efficiently process, manipulate and visualise large datasets (Arden University, 2024). Also Python has a lot of specialised libraries that can simplify complex analytical tasks and improve the productivity of data analytics environments.

A number of Python libraries and tools were used to prepare, process, analyse and visualise the sales dataset in this project.

Google Colaboratory

Google Colab was used to execute the Python code for this project. It is a cloud based interactive notebook environment that lets the users write and execute the code through a web browser without the installation of complex local software. It is very useful for data analytics especially because it integrates code execution, outputs, visualisations and explanatory texts in a single document.

Interactive notebook environments such as Jupyter Notebook and Google Colab are important tools in data science workflows because they support experimentation, reproducibility and step by step analytical development. For many Python libraries used in data science Google Colab also provides built in support which makes it very suitable for both advanced and beginner users.

How Google Colab was used in this project:

- To upload and access sales dataset.
- To execute Python code.
- To show outputs and visualisations.
- The Documentation of the analytical process.

Pandas

Pandas is an open source library from Python used for efficient data manipulation and analysis. Important data structures such as Series and DataFrames are provided by Pandas, which allows structured datasets to be

organised and processed efficiently. It is used mainly because it simplifies data cleaning, filtering, aggregation and transformation procedures.

How Pandas was used in this project:

- To import the sales dataset.
- To inspect the structure of the dataset.
- To handle the missing values.
- To remove the duplicate records.
- To transform the variables.
- To perform exploratory data analysis.

Pandas is one of the most essential libraries for modern data analytics due to its high efficiency when working with structured datasets. (McKinney, W. 2022)

NumPy

It is a numerical computing library which is used to perform mathematical and statistical operations on arrays and datasets. It forms the foundation for many scientific and analytical Python libraries (Arden University 2024). It is very useful when working with large numerical datasets and improves the computational performance.

How NumPy was used in this project:

- For numerical calculations.
- For statistical analysis.
- For data transformation.
- For outlier detection procedures.

Matplotlib

It is a data visualisation library from Python which is used to create charts, graphs and plots. Visualisation is an important stage within data analytics because graphical representation helps analysts identify patterns, trends and relationships more effectively than raw numerical data alone (Arden University 2024).

Matplotlib visualisation techniques:

- Line graphs.
- Bar charts.
- Scatter plots.
- Histograms.
- Pie charts.

Matplotlib was used in this project to create visual representation of the sales dataset from the above techniques whenever it was necessary to help support better data interpretation and analysis.

All together the tools and technologies mentioned above helped to create an efficient environment for preparing, analysing and visualising the sales dataset during the entire project.

3. DATASET DESCRIPTION

The sales dataset contains transactional information related to customer orders, shipping methods, sales personnel, sales value etc. The dataset has seven attributes.

Dataset attributes are as follows:

- order_id – Unique identification number assigned to each order.
- customer_id – Unique identification number assigned to each customer.
- sales_person – Name of the sales person responsible for the transaction.
- Priority – Priority level assigned to the order (high, medium, low, critical).
- ship_mode – Shipping method used for delivering the order.
- date – Date on which the order was placed.
- value_£ - Monetary sales value of the transaction.

Among these attributes the main attributes of interest for the analysis are

- sales_person – The project requires an investigation into the sales performance of different sales people over the given time period. So this variable is used to compare total sales value between salesperson and find out who generated the highest or lowest sales
- value_£ - This attribute is important in identifying the revenue generation patterns of the company, sales distribution and overall business performance.
- Date – This attribute is important as the aim of the project is to make exploratory analysis on the given dataset over the given time period of the dataset.
- Priority – This attribute is important as the project demands an investigation into finding out if a relationship exists between the priority level and sales value. By doing so it may be possible to identify whether high priority transactions generate higher sales value.

Overall the aim of the analysis is to find out possible sales trends and support future sales performance predictions for the sales representatives.

4. ANALYSIS OF THE SALES DATASET

The following parts of the report shows the implementation of the above mentioned tools, technologies and methods to analyse the given sales dataset with justification as to why these methods were relevant for the investigation. The analysis is done with a focus on “sales person” and “Total sales value” over the time period of the given data. One of the aims of the project is to identify whether there is a correlation between “priority” and “value” over the same time period. The main focus of this project is to use analytical methods to help predict possible sales trends in the future for the identified sales representatives. Each section

4.1 Data Understanding

The first step is to understand the structure, content and quality of the sales data. This is the first step in the data analytics workflow where the data must first be inspected before cleaning, transformation, analysis and visualisation.

Importing the required libraries and sales dataset

- Import command - The libraries Pandas, NumPy and Matplotlib were imported. Pandas to import and inspect the dataset. NumPy for numerical operations and Matplotlib for later data visualisations.
- `pd.read_excel()` – the command used to import sales dataset. The imported data is stored in a dataframe called `df`

Viewing the dataset

- `df.head()` – This command was used to display the first five rows of the dataset, which helps to make sure that the data has been imported correctly and that all columns are visible.
- `df.tail()` – This command was used to display the last five rows of the dataset, which helps to make sure that the structure remained consistent till the end.
- `df.sample(5)` – This command was used to display any five random rows, which helps in a quick inspection of the dataset from the middle.

Checking the size and structure of the dataset

- `df.shape` – This command was used to identify the number of rows and columns in the dataset, which helps to understand whether the dataset was uploaded completely.
- `df.columns` – This command was used to display all the column names.
 - ❖ **Adjustment:** During the first import, the columns appeared as `Index(['Unnamed: 0', 'Unnamed: 1', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4', 'Unnamed: 5', 'Unnamed: 6'], dtype='object')`. This is because the first row of the dataset was empty. This was resolved by re importing the dataset using the parameter `header=1`.
- `df.index` – This command was used to inspect the row indexing of the dataframe, which helps in confirming that the dataset was imported correctly and that the row indexing structure is continuous without interruption.

Checking the datatypes

- `df.info()` – This command was used to show a summary of the dataframe which includes the column names, non null values and datatypes. The output showed there were no missing values and the datatypes were appropriate.
- `df.dtypes` – This command was used to check the datatype of each individual column.
 - ❖ **Adjustment:** The `value_£` attribute was renamed to `sales_value` so that it aligns more with the rest of the attribute names and also makes coding less confusing.
 - ❖ **Adjustment:** The date column was initially stored as a full datetime variable which also had time information. Since the investigation mainly focuses on sales over time than specific timestamps the datetime values were normalised only to retain the date part but the datetime format was preserved.

Initial statistical summary

- `df.describe()` – This command was used to create an initial descriptive statistics view for the date and numerical attributes. The output showed the dataset has 1001 rows and they are the sales transactions from 12 June 2020 to 30 November 2020. The mean of the `sales_value` is 1778.402248, median is 417.580000, minimum value is 2.450000 and

maximum value is 27448.240000. The huge difference between mean and median shows that the sales values are unevenly distributed and may have high sales_value transactions that affects the average value. This shows that outlier detection should be carried out in the further steps of the investigation.

Checking unique values

- `df.nunique()` – This command was used to identify the number of unique values in each columns. The output showed that the dataset contained 3 sales representatives, 5 priority categories, 3 shipping modes, 437 customers and 165 transaction dates. The output also shows that `order_id` had lesser unique values than total rows, which shows some orders are repeating. This should be investigated further.

Checking missing values

- `df.isnull().sum()` – This command was used to check if there were any missing values in each column and the output showed no missing values.
- `df.notnull().sum()` – This command was used to check how many non missing values existed in each column and the output showed 1001, which means there are no missing values in all of the 1001 rows.

Checking duplicate records

- `df.duplicated().sum()` – This command was used to check for duplicate records in the dataset, the output showed 0, indicating there are no duplicate rows in the dataset.

Category distribution analysis

- `value_counts()` – This function was applied to sales_person, priority and ship_mode attributes of the dataset for better understanding of the categorical variables. The output shows there are 3 salesperson with Peter having the highest number of transactions. The priority attribute had 5 categories from Low to Critical and one category named Not Specified which need further investigation. The ship_mode attribute had 3 categories with “plane” being the most used shipping mode.

Moreover the sum of all these `value_counts()` of different categories gives 1001, showing that every row in the dataset belongs to a valid category.

Summary

The data understanding part made sure that the sales dataset was imported successfully and included all the 1001 rows with 7 attributes. The initial statistical analysis showed that the sales data covered the period from June 2020 to November 2020 and the sales value changed significantly, indicating that there may be possible high value transactions. Moreover further analysis showed that no missing values or duplicate rows exists. Now the dataset is suitable for further exploratory investigation.

4.2 Data Preparation

This is an important step in data analysis process as raw data may contain inconsistencies, formatting problems, missing values etc and these problems might affect the accuracy of the analysis. So during data preparation stage the data is cleaned, transformed and organised into a reliable format that is suitable for further analysis.

4.2.1 Data Cleaning

Creating a working copy

- `copy()` – This function was used to create a separate copy of the dataset named `df_clean`. This makes sure that the original dataset remains unchanged.

Duplicate check

- `duplicated().sum()` – This function was used again similar to the previous section to check for duplicate values and the output showed that the dataset contains 0 duplicate values.

Column names check

- `df.columns` – This command was used again to study the column names. The output showed that the column names are already consistent, readable, all in lower case and ready for analysis. So no further cleaning is required for the column names.

4.2.2 Handling Missing Data

Missing values check

- `isnull().sum()` – This function was used again to check for missing values, the output showed none.
- `notnull().sum()` – This function was also used to verify the number of valid rows and the output returned 1001 which corresponds to all the rows in the dataset.

4.2.3 Data Transformation

Analysing datatypes

- `convert_dtypes()` – This function was used to convert the dataset attributes into more suitable pandas datatypes. Initially some attributes like `customer_id`, `sales_person`, `priority` and `ship_mode` were stored as generic object types and now they are converted to string datatype.
- `Info()` – This function was used again to make sure that the dataset datatypes were updated.

Inspecting column contents

- `value_counts()` – This function was used to see how many times each category appeared under the attributes in the dataset. The output showed that the categories were consistent. The `priority` attribute had a category called `Not Specified` and it appears 213 times. Since it is a large number those records should not be deleted as they are valid sales data, also they cant be replaced with `Low`, `Medium`, `High` or `Critical` because that would mislead the analysis. Hence `Not Specified` is treated as another valid category.
- `df_clean[col].str.strip()` – This function was used to remove any unnecessary spaces before or after text values while keeping the original format of the dataset.

4.2.4 Outlier Detection

Outlier detection was performed on the numerical variables using the IQR (Interquartile Range) method. Q1 and Q2 were calculated for each numerical

columns and the IQR range was used to find lower and upper boundaries to identify unusual values.

Code for calculating Q1, Q3 and IQR:

```
Q1 = df_clean[col].quantile(0.25)
```

```
Q3 = df_clean[col].quantile(0.75)
```

```
IQR = Q3 - Q1
```

Code to define lower and upper boundaries to identify potential outliers:

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

Code to identify values that falls outside the IQR:

```
mask = (df_clean[col] < lower) | (df_clean[col] > upper)
```

Code to create a boxplot to visually understand the distribution of the numerical variables:

```
df_clean.select_dtypes(include=np.number).plot(kind='box')
```

The outlier detection found out that there could be 122 potential outliers in the sales_value attribute. Even though many high sales values were considered as statistical outliers by the IQR method, these values are retained within the dataset. This is because the dataset represents real business sales transactions, where large sales values may represent real high value orders than data errors. Removing them may reduce the accuracy of the insights that we receive after the data analysis.

4.2.5 Data Reduction and Feature Preparation

A final dataset named `df_analysis` was created by copying the clean dataset. To help the analysis more extra date based variables were extracted from the date column using pandas datetime functions. Three new variables were created: year, month and day.

This helps in time based analysis like monthly sales trends, seasonal sales trends and daily transactional analysis.

Code to extract the year component from the date attribute

```
df_analysis['year'] = df_analysis['date'].dt.year
```

Code to extract the month component from the date attribute

```
df_analysis['month'] = df_analysis['date'].dt.month
```

Code to extract the day component from the date attribute

```
df_analysis['day'] = df_analysis['date'].dt.day
```

Summary

The final sales dataset contains 1001 rows with no missing values and correctly formatted variables. There were several high value transactions that were identified as statistical outliers but they were kept unchanged because they represent genuine business sales activity. After the preparation process the sales dataset became more reliable, consistent and ready for further analysis and visualisation.

4.3 Data Analysis

Data analysis helps in identifying patterns, trends and relationships within the cleaned dataset. To analyse sales trends based on salesperson, priority levels, shipping modes and monthly trends methods such as descriptive statistics, grouping operations and visualisations were used.

4.3.1 Descriptive Analysis

Generating summary statistics

- `describe()` – This function was used on the `sales_value` attribute to summarise the overall sales performance of the prepared dataset.
- `Sum()`, `mean()`, `min()`, `max()` – These functions were used on the `sales_value` attribute to have an initial understanding of the sales distribution.
- `count()` – This function was used on the `order_id` attribute to identify the total number of orders.

The dataset contains 1001 completed orders with a total sales value of £1.78 million. The average sales value was around £178 per transaction, while minimum and maximum were £2.45 and £27,448.24. This shows a huge variation in sales values across the dataset.

The median value (£417.58) was lower than the mean value (£1778.40) which shows that the sales distribution was positively skewed (we have made this observation in the above outlier detection section as well), which means a small number of high value transactions increase the overall average sales value.

4.3.2 Salesperson Performance Analysis

Calculating total sales for each person

```
df_analysis.groupby('sales_person')['sales_value'].sum()
```

Calculating average sales value per transaction for each sales person

```
df_analysis.groupby('sales_person')['sales_value'].mean()
```

Counting the number of orders handled by each salesperson

```
df_analysis.groupby('sales_person')['order_id'].count()
```

Creating a bar chart to compare total sales by sales person

```
salesperson_total.plot(kind='bar')
```

The outputs showed that Pryia made the highest total sales value, which is £672,228.97. Following Pryia is Kelly with a total sales value of £558,659.62 and Peter with £549,292.06. Pryia also has the highest average sales value per transaction which is around £1,988.84.

The highest number of orders were handled by Peter which is 344 transactions, while Pryia handled 338 transactions. But still the highest revenue was generated by Pryia. This indicates that the transactions handled by Pryia were high value transactions compared with the others.

4.3.3 Priority Based Sales Analysis

Calculating total sales value for each priority

```
df_analysis.groupby('priority')['sales_value'].sum()
```

Calculating average sales value for each priority

```
df_analysis.groupby('priority')['sales_value'].mean()
```

Counting the number of orders in each priority

```
df_analysis.groupby('priority')['order_id'].count()
```

Creating a bar chart to compare total sales by priority

```
priority_total.plot(kind='bar')
```


The outputs shows that Critical orders generate the highest total sales value of £403,652.33. Then comes High priority orders with a total sales value of £381,543.40. Low priority orders generated the lowest sales value which is £292,535.81.

However while checking the average, it is clear that High priority orders had the highest average sales value which was around £1,946.65. Low priority orders had the lowest average sales value at around £1,462.68. This shows that although Critical orders produce the most revenue, it is mostly the High priority orders that gives higher value per transaction.

The Not Specified category had 213 orders and generated £372,111.74 in total sales.

Relationship between priority and sales value:

The generated box plot shows that sales value distribution is similar for different priority categories. The median sales value and interquartile range overlapping shows that priority does not have a very strong influence on sales value. But the categories High and Medium priority has a little bit more average sales value compared to the other categories. But along with that all the priority categories has many high value outliers, which means that very sales can happen regardless of the assigned priority level. So the analysis shows that there is only a weak relationship between priority and sales value.

4.3.4 Shipping Mode Analysis

Calculating the total sales value for each shipping mode

```
df_analysis.groupby('ship_mode')['sales_value'].sum()
```

Calculating average sales value per order for each shipping mode

```
df_analysis.groupby('ship_mode')['sales_value'].mean()
```

Counting the number of orders under each shipping mode

```
df_analysis.groupby('ship_mode')['order_id'].count()
```

Creating a bar chart to compare total sales by shipping mode

```
ship_total.plot(kind='bar')
```

The output shows that Plane generated the highest total sales value which is £1,018,909.07. Lorry generated the second highest sales value of £627,825.97 followed by Van with £133,445.61. Also the Plane was the most frequently used shipping mode with 770 orders. So that is why it was the highest contributor to the total sales value. However Lorry had the highest average sales value per order at around £4,982.75 even though it only had 126 orders. This also shows that even though Lorry deals with just a few orders they might be high value orders.

4.3.5 Time Based Analysis

Calculating total sales value for each month

```
df_analysis.groupby('month')['sales_value'].sum()
```

Counting the number of orders in each month

```
df_analysis.groupby('month')['order_id'].count()
```

Calculating the average sales value per order for each month

```
df_analysis.groupby('month')['sales_value'].mean()
```

Creating a line chart to show how total sales changes across months

```
monthly_sales.plot(kind='line', marker='o')
```

The outputs showed that July generated the highest sales with a total of 378,143.61 followed by October with 355,178.46. June had the lowest sales value which was 144,279.72. The highest number of orders were recorded in August with 193 transactions while June had the lowest number of orders. But the highest average sales value happened in October at around 2,126.82 per order which shows that October had lesser orders than August but they were

high value transactions on average. The line graph also shows a sharp increase in sales from June to July and a small decline in August and September. Then it increases again in October and decreases in November. Overall the highest sales activity was in July and October.

Summary

The analysis showed that sales performance changed across different categories. Pryia generated the highest total sales among sales persons, Plane was the most commonly used shipping mode and contributed to the highest sales value. Priority level just had a weak relationship with sales value. Also time based analysis showed that sales were higher during July and October and sales performance varied monthly.

4.4 Data Visualisation

Data visualization was performed to better understand the sales dataset through graphical representations. Different visualization techniques were used to identify distributions, trends, comparisons, outliers and relationships between variables using Matplotlib and Seaborn libraries.

4.4.1 Histogram

Distribution of sales value

A histogram was created using `plt.hist()` to visualise the distribution of sales values. The output shows that most of the sales values happens at low ranges while just a few orders have very high values.

4.4.2 Bar Charts

Monthly order count

A bar chart was created using `plt.bar()` to see the number of orders in different months. The output shows that months 7 and 8 has highest number of orders.

Total sales by salesperson

A bar chart was created using `plt.bar()` to see the total sales generated by each salesperson. The output shows differences in sales contribution between the salesperson.

Total sales by priority

A bar chart was created using `plt.bar()` to see the total sales across different priority levels. The output shows that critical and High priority orders made higher sales value compared to the other priority categories.

Total sales by shipping mode

A bar chart was created using `plt.bar()` to see the total sales by shipping mode. The output shows that plane shipping was the most frequently used and it generated highest sales value.

4.4.3 Box Plots

Boxplot of sales value

A box plot was created using `plt.boxplot()` to understand the spread and outliers in sales values. The output shows the presence of multiple high value outliers.

Sales value distribution by priority

A grouped boxplot was created using `DataFrame.boxplot()` to compare sales value distribution across priority categories. The output shows that variations in sales was spread between priorities.

4.4.4 Scatter Diagram

Sales value by order

A scatter plot was created using `plt.scatter()` to visualise sales values across order IDs. The output shows that there are a lot of high value transactions and also shows the variability of sales values.

4.4.5 Pie Charts

Order distribution by shipping mode

A pie chart was created using `plt.pie()` to display the proportion of orders across shipping modes. The largest share of orders were made by plane shipping.

Sales contribution by salesperson

A pie chart was created using `plt.pie()` to show the percentage contribution of each salesperson to total sales. The output shows that the contributions were relatively balanced.

4.4.6 Line Diagram

Monthly sales trend

A line chart was created using `plt.plot()` to understand the sales trend across months. The output shows changes in monthly sales value over time.

4.4.7 Correlation Heatmap Analysis

A correlation heatmap was created using `sns.heatmap()` to analyse the correlation between numerical variables. The output shows that there is almost no relationship between month and sales value.

A similar correlation heatmap was created to analyse the correlation between priority and sales value. Different priorities were first converted into numerical values using a mapping technique to create the heatmap. The output showed a correlation value of approximately 0.026 which indicates a very weak positive correlation, meaning that priority has little influence on sales value.

Summary

The visualisations were helpful in understanding the overall sales patterns, monthly trends, category-wise comparisons and relationships within the dataset. The analysis showed variations in sales distribution, differences between categories, and weak correlations between certain variables.

5. CONCLUSION

5.1 Results and Discussions

The sales analysis showed the total sales value varied significantly across orders and categories. The histogram and boxplot visualisations showed that most sales transactions were concentrated in lower sales ranges, with only a few transactions having very high sales values. The presence of several outliers in the boxplot confirmed the fact that a few orders contributed disproportionately to total revenue. These patterns are common in commercial datasets where high-value customers or bulk purchases significantly impact business revenue (Han, 2023).

The monthly sales trend analysis showed fluctuations in sales performance. July had the highest total sales while June and November had lower sales values. The line chart and monthly order count chart showed that after July customer activity was relatively stable, but revenue generation changed from month to month. This suggests that average order value, not just the number of orders, influenced monthly revenue performance.

The analysis of the salespersons showed that Pryia was responsible for the highest percentage of total sales followed by Kelly and Peter. The salespersons' contribution is relatively even, that means the sales result is stable in the company. But Pryia's slightly higher contribution indicates that some sales strategies or customer management techniques may have been more effective and can be further analysed for business improvement.

The shipping mode analysis shows that Plane shipping is the most used shipping mode based on the number of orders and sales value. The pie chart shows that Plane shipping covers about 77% of all shipping activities meanwhile Lorry and Van contribute to much smaller portions. This means that customers or the company prefer faster delivery modes even when the shipping costs are higher. These findings can help the company to improve logistics planning and resource allocation.

The priority-based analysis shows a very weak correlation between sales value and priority level as seen in the correlation heatmap. The correlation value is close to zero which means that higher priority orders do not necessarily lead to higher sales revenue. The sales value by order scatter plot visualisation showed that sales transactions were very scattered and it had little linear correlation between order sequence and sales value. The majority of transactions remained

within lower sales ranges and only a few outlier high-value orders stood out. This reflects the uneven distribution often seen in sales datasets.

Overall, the results show how data analytics and visualisation techniques can support business decision making by analysing the dataset to identify patterns, trends, and operational behaviours. Interpretability was enhanced through visualisation techniques like histograms, boxplots, scatter plots, pie charts, line charts and heatmaps which helped in converting raw data into actionable insights.

Also according to the investigation, if the current trend is followed, Pryia is likely to continue being the strongest sales representative in the future periods. He/She made the highest total sales and the highest average sales value per order, which means that his/her future sales trend may continue being higher than the other representatives.

Kelly most probably will be in a stable middle position as his/her total sales and average sales value were close to Peter's but a bit higher in total. This means that Kelly may continue providing consistent sales performance although not at the same level as Pryia. Peter dealt with the highest number of orders but generated the lowest total and average sales among the three representatives. This shows that Peter's future sales path may continue to be the same way, implying that he might still process numerous orders but with lower average order values unless there is an enhancement in his sales strategy.

Overall, the anticipated sales trend indicates that Pryia is likely to maintain her lead in revenue generation, Kelly may remain stable, and Peter might need assistance to elevate the value of each transaction. The company could utilize Priya's sales methodology as a standard and offer targeted training to enhance the average sales value across the team.

5.2 Reflective Report

One of the significant strengths of the investigation was the methodical data preparation process. The dataset underwent thorough examination for duplicate entries, missing values, inconsistent categorical variables, datatype discrepancies, and outliers prior to the analysis. This preprocessing phase bolstered the reliability and consistency of the dataset, which is regarded as a crucial step in professional data analytics workflows (McKinney, 2022). The

application of pandas functions such as `isnull()`, `groupby()`, `describe()`, and datatype conversion techniques facilitated efficient management and exploration of the dataset.

The investigation successfully uncovered several valuable business insights. Nevertheless, several limitations were recognized during the investigation. Firstly, the dataset contained only a restricted time frame within a single year. Consequently, long-term seasonal trends and annual sales forecasting could not be thoroughly assessed. A more extensive dataset including the data of several years would enhance the accuracy of trend predictions and facilitate the application of more sophisticated forecasting methods.

Additionally, the analysis primarily concentrated on descriptive and exploratory analytics. Although these approaches successfully uncovered patterns and correlations predictive analytical methods such as regression analysis, time-series forecasting, or machine learning models were not utilized. Future research could implement forecasting models like Linear Regression, ARIMA, or Prophet to more precisely forecast future sales performance (Siebert et al., 2021).

Another setback is that numerous potentially significant business variables were absent from the dataset. Attributes such as customer demographics, product categories, discounts, regional sales data, and customer satisfaction metrics could help produce deeper business insights if incorporated in subsequent studies. The inclusion of additional variables would enable more sophisticated multivariate analysis and customer behavior modeling.

Future work could concentrate on the implementation of predictive analytics and interactive dashboards utilizing tools such as Plotly or Power BI. The integration of automated reporting systems and real-time business monitoring could also be incorporated to enhance operational decision-making. Furthermore, clustering techniques could be employed to more effectively identify customer segments or patterns in salesperson performance.

To summarise, this investigation showed that analytical methods and visualization strategies are extremely useful for understanding sales patterns and helps in business decision-making. Furthermore the investigation emphasized the significance of data preparation, critical analysis, and methodological assessment in the data analytics process.

6. REFERENCES

- Arden University (2025) Programming for Data Analytics – Lessons.
- Joshi, A. and Tiwari, H. (2023) - An Overview of Python Libraries for Data Science, Journal of Engineering Technology and Applied Physics.
- Kabir, M.A., Ahmed, F., Islam, M.M. and Ahmed, M.R. (2024) - Python for Data Analytics: A Systematic Literature Review of Tools, Techniques and Applications.
- Google Research (2025) - Google Colaboratory Documentation.
- McKinney, W. (2022) - Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter. 3rd edn. Sebastopol: O'Reilly Media.
- The pandas development team (2025) - pandas documentation.
- Matplotlib Development Team (2025) - Matplotlib Documentation.
- Martins, P. et al. (2025) - A Benchmark on Large Real-World Datasets.
- Zuo, K. (2026) - Pandas and Matplotlib: A Case Study of the Titanic Dataset.
- Han, S. (2023) - Mastering Data Visualization with Python: Practical Tips for Beginners.
- Sial, A. H. et al. (2021) - Data Visualization using Matplotlib and Seaborn Libraries in Python. International Journal of Advanced Trends in Computer Science and Engineering.
- Molin, S. (2021) - Hands-On Data Analysis with Pandas. Packt Publishing.
- Siebert, J. et al. (2021) - A Systematic Review of Python Packages for Time Series Analysis

7. APPENDIX

Data Understanding

RAM ☐ Disk ☐

Data Understanding

```
[1]
✓ Os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
```

```
[2]
✓ Os
df = pd.read_excel('Sales Data_PDA_4052.xlsx')
```

```
[3]
✓ Os
df.head()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6
0	order_id	customer_id	sales_person	priority	ship_mode	date	value_£
1	8999	C406	Kelly	High	Plane	2020-06-12 00:00:00	1553.44
2	47872	C100	Kelly	Critical	Plane	2020-06-12 00:00:00	315.4
3	29988	C006	Kelly	High	Lorry	2020-06-12 00:00:00	5075.89
4	37348	C492	Kelly	High	Plane	2020-06-13 00:00:00	520.67

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
[4]
✓ Os
df.tail()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6
997	12258	C440	Pryia	Low	Plane	2020-11-30 00:00:00	2417.14
998	57061	C234	Pryia	Low	Plane	2020-11-30 00:00:00	451.18
999	57061	C234	Pryia	Low	Plane	2020-11-30 00:00:00	35.15
1000	57061	C234	Pryia	Low	Plane	2020-11-30 00:00:00	50.14
1001	36416	C691	Pryia	Low	Plane	2020-11-30 00:00:00	127.18

```
[5]
✓ Os
df.sample(5)
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6
796	46469	C006	Pryia	Critical	Plane	2020-08-16 00:00:00	85.16
664	355	C335	Pryia	Low	Plane	2020-06-13 00:00:00	1375.84
140	47974	C287	Kelly	Medium	Plane	2020-08-31 00:00:00	16.51
793	29346	C353	Pryia	Low	Plane	2020-08-16 00:00:00	20.84
249	56995	C682	Kelly	Medium	Van	2020-10-15 00:00:00	142.58

```
[6]
✓ Os
df.shape
```

(1002, 7)

```
[7]
✓ Os
df.columns
```

Index(['Unnamed: 0', 'Unnamed: 1', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4', 'Unnamed: 5', 'Unnamed: 6'], dtype='object')

```
[9]
✓ Os
df = pd.read_excel("Sales Data_PDA_4052.xlsx", header=1)
```

+ Code + Text

```
[10]
✓ Os
df.columns
```

Index(['order_id', 'customer_id', 'sales_person', 'priority', 'ship_mode', 'date', 'value_£'], dtype='object')

```
[14]
✓ Os
df.index
```

RangeIndex(start=0, stop=1001, step=1)

```
[11]
✓ Os
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001 entries, 0 to 1000
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  --
0   order_id    1001 non-null   int64
1   customer_id 1001 non-null   object
2   sales_person 1001 non-null   object
3   priority     1001 non-null   object
4   ship_mode    1001 non-null   object
5   date         1001 non-null   datetime64[ns]
6   value_£      1001 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1), object(4)
memory usage: 54.9+ KB
```

```
[12]
✓ Os
df.dtypes

order_id    int64
customer_id object
sales_person object
priority     object
ship_mode    object
date         datetime64[ns]
value_£      float64
dtype: object

[15]
✓ Os
df.rename(columns={'value_£': 'sales_value'}, inplace=True)

[16]
✓ Os
df.columns

Index(['order_id', 'customer_id', 'sales_person', 'priority', 'ship_mode', 'date', 'sales_value'], dtype='object')
```

```
[17]
✓ Os
df.describe()

...      order_id      date  sales_value
count  1001.000000      1001    1001.000000
mean   30231.488511  2020-09-04 08:03:21.398601216  1778.402248
min     224.000000    2020-06-12 00:00:00      2.450000
25%    16451.000000    2020-07-26 00:00:00    134.400000
50%    29667.000000    2020-09-04 00:00:00    417.580000
75%    44583.000000    2020-10-15 00:00:00   1816.930000
max     59591.000000    2020-11-30 00:00:00  27448.240000
std     16744.674540              NaN    3457.457206

[18]
✓ Os
df['date'] = pd.to_datetime(df['date']).dt.normalize()
```

```
[19]
✓ Os
df.describe()

...      order_id      date  sales_value
count  1001.000000      1001    1001.000000
mean   30231.488511  2020-09-04 08:03:21.398601216  1778.402248
min     224.000000    2020-06-12 00:00:00      2.450000
25%    16451.000000    2020-07-26 00:00:00    134.400000
50%    29667.000000    2020-09-04 00:00:00    417.580000
75%    44583.000000    2020-10-15 00:00:00   1816.930000
max     59591.000000    2020-11-30 00:00:00  27448.240000
std     16744.674540              NaN    3457.457206
```

```
[22] df.head()
✓ 0s
```

	order_id	customer_id	sales_person	priority	ship_mode	date	sales_value
0	8999	C406	Kelly	High	Plane	2020-06-12	1553.44
1	47872	C100	Kelly	Critical	Plane	2020-06-12	315.40
2	29988	C006	Kelly	High	Lorry	2020-06-12	5075.89
3	37348	C492	Kelly	High	Plane	2020-06-13	520.67
4	28001	C231	Kelly	Critical	Plane	2020-06-14	8431.37

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
[24] df['date'].max()
✓ 0s
Timestamp('2020-11-30 00:00:00')
```

```
[23] print("Earliest date:", df['date'].min().date())
print("Latest date:", df['date'].max().date())
✓ 0s
Earliest date: 2020-06-12
Latest date: 2020-11-30
```

```
[25] df.nunique()
✓ 0s
```

order_id	645
customer_id	437
sales_person	3
priority	5
ship_mode	3
date	165
sales_value	998

dtype: int64

```
[26] df.isnull().sum()
✓ 0s
```

order_id	0
customer_id	0
sales_person	0
priority	0
ship_mode	0
date	0
sales_value	0

dtype: int64

```
[27] df.isnull().sum()
✓ 0s
```

```
[27] df.notnull().sum()
✓ 0s
```

order_id	1001
customer_id	1001
sales_person	1001
priority	1001
ship_mode	1001
date	1001
sales_value	1001

dtype: int64

```
[28] df.duplicated().sum()
✓ 0s
np.int64(0)
```

```
[29] ✓ 0s df['sales_person'].value_counts()
...
count
sales_person
Peter      344
Pryia      338
Kelly      319
dtype: int64

[30] ✓ 0s df['priority'].value_counts()
...
count
priority
Critical    221
Not Specified 213
Low          200
High         196
Medium       171
dtype: int64
```

```
dtype: int64

[31] ✓ 0s df['ship_mode'].value_counts()
...
count
ship_mode
Plane      770
Lorry       126
Van         105
dtype: int64
```

Data Preparation

Data Preparation

```
[ ] df_clean = df.copy()
```

```
[ ] df_clean.head()
```

	order_id	customer_id	sales_person	priority	ship_mode	date	sales_value
0	8999	C406	Kelly	High	Plane	2020-06-12	1553.44
1	47872	C100	Kelly	Critical	Plane	2020-06-12	315.40
2	29988	C006	Kelly	High	Lorry	2020-06-12	5075.89
3	37348	C492	Kelly	High	Plane	2020-06-13	520.67
4	28001	C231	Kelly	Critical	Plane	2020-06-14	8431.37

Next steps: [Generate code with df_clean](#) [New interactive sheet](#)

```
[ ] df_clean.duplicated().sum()
```

```
np.int64(0)
```

```
[ ] df_clean.columns
```

```
[ ] df_clean.columns
```

```
Index(['order_id', 'customer_id', 'sales_person', 'priority', 'ship_mode', 'date', 'sales_value'], dtype='object')
```

```
[ ] df_clean.isnull().sum()
```

	0
order_id	0
customer_id	0
sales_person	0
priority	0
ship_mode	0
date	0
sales_value	0

dtype: int64

```
[ ] missing_total = df_clean.isnull().sum().sum()
```

```
missing_total
```

```
np.int64(0)
```

```
[ ] df_clean.notnull().sum()
```

	0
order_id	1001
customer_id	1001
sales_person	1001
priority	1001
ship_mode	1001
date	1001
sales_value	1001

dtype: int64

```
[ ] df_clean.dtypes

...
order_id      int64
customer_id    object
sales_person   object
priority       object
ship_mode      object
date          datetime64[ns]
sales_value    float64
dtype: object

[ ] df_clean = df_clean.convert_dtypes()
```

```
[ ] df_clean.dtypes

...
order_id      Int64
customer_id    string[python]
sales_person   string[python]
priority       string[python]
ship_mode      string[python]
date          datetime64[ns]
sales_value    Float64
dtype: object
```

```
[ ] df_clean.info()

...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001 entries, 0 to 1000
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   order_id    1001 non-null  Int64
1   customer_id 1001 non-null  string
2   sales_person 1001 non-null  string
3   priority     1001 non-null  string
4   ship_mode    1001 non-null  string
5   date        1001 non-null  datetime64[ns]
6   sales_value  1001 non-null  Float64
dtypes: Float64(1), Int64(1), datetime64[ns](1), string(4)
memory usage: 56.8 KB
```

```
[1] for col in df_clean.select_dtypes(include=['string']).columns:
      print("column:", col)
      print(df_clean[col].value_counts(dropna=False))
      print("-" * 50)
```

```
... Column: customer_id
customer_id
C475    11
C191     9
C733     8
C497     8
C737     8
..
C795     1
C313     1
C598     1
C083     1
C655     1
Name: count, Length: 437, dtype: Int64
-----
Column: sales_person
sales_person
Peter    344
Pryia    338
Kelly    319
Name: count, dtype: Int64
-----
Column: priority
priority
..
..
```

```
...
C795     1
C313     1
C598     1
C083     1
C655     1
Name: count, Length: 437, dtype: Int64
-----
Column: sales_person
sales_person
Peter    344
Pryia    338
Kelly    319
Name: count, dtype: Int64
-----
Column: priority
priority
Critical    221
Not Specified 213
Low         200
High        196
Medium      171
Name: count, dtype: Int64
-----
Column: ship_mode
ship_mode
Plane      770
Lorry      126
Van         105
Name: count, dtype: Int64
-----
```

```
[1] for col in df_clean.select_dtypes(include=['string']).columns:
      df_clean[col] = df_clean[col].str.strip()
```

```
[1] for col in df_clean.select_dtypes(include=['string']).columns:
      print("column:", col)
      print(df_clean[col].value_counts(dropna=False))
      print("-" * 50)
```

```
... Column: customer_id
customer_id
C475    11
C191     9
C733     8
C497     8
C737     8
..
C795     1
C313     1
C598     1
C083     1
C655     1
Name: count, Length: 437, dtype: Int64
-----
Column: sales_person
sales_person
Peter    344
Pryia    338
Kelly    319
..
..
```



```

C313    1
C598    1
C083    1
C655    1
Name: count, Length: 437, dtype: Int64

```

```

-----
Column: sales_person
sales_person
Peter    344
Pryia    338
Kelly    319
Name: count, dtype: Int64

```

```

-----
Column: priority
priority
Critical    221
Not Specified 213
Low         200
High        196
Medium      171
Name: count, dtype: Int64

```

```

-----
Column: ship_mode
ship_mode
Plane    770
Lorry    126
Van       105
Name: count, dtype: Int64

```

```

df_clean['sales_value'].head(10)

```

```

sales_value
0    1553.44
1     315.4
2    5075.89
3     520.67
4    8431.37
5    6976.84
6     143.95
7     103.25
8    2829.17
9     141.59

```

```

dtype: Float64

```

Outlier Detection

```

df_clean.describe()

```

	order_id	date	sales_value
count	1001.0	1001	1001.0
mean	30231.488511	2020-09-04 08:03:21.398601216	1778.402248
min	224.0	2020-06-12 00:00:00	2.45
25%	16451.0	2020-07-26 00:00:00	134.4
50%	29667.0	2020-09-04 00:00:00	417.58
75%	44583.0	2020-10-15 00:00:00	1816.93
max	59591.0	2020-11-30 00:00:00	27448.24
std	16744.67454	NaN	3457.457206

```
[ ]
outlier_summary = {}

for col in df_clean.select_dtypes(include=np.number).columns:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    mask = (df_clean[col] < lower) | (df_clean[col] > upper)

    outlier_summary[col] = mask.sum()

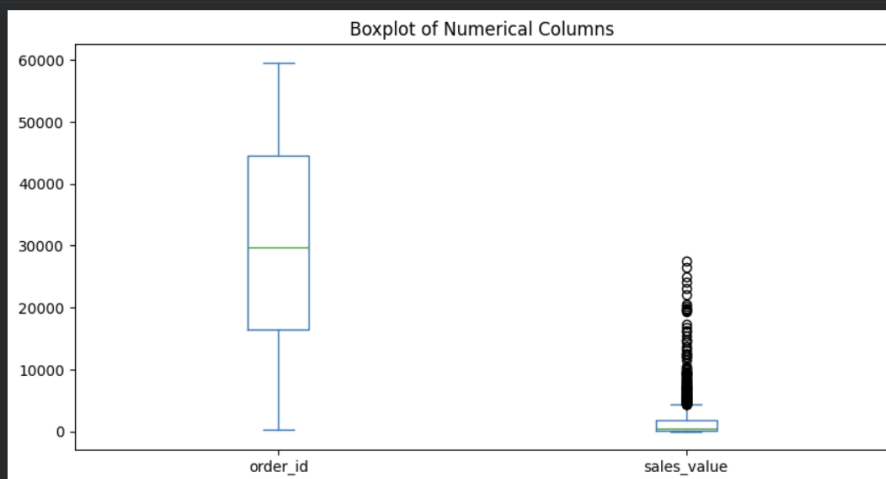
outlier_summary

{'order_id': np.int64(0), 'sales_value': np.int64(122)}
```

```
[ ]
df_clean.select_dtypes(include=np.number).plot(
    kind='box',
    figsize=(10,5)
)

plt.title("Boxplot of Numerical Columns")
plt.show()
```

```
[ ]
plt.title("Boxplot of Numerical Columns")
plt.show()
```



Data reduction

```
[ ]
df_analysis = df_clean.copy()

[ ]
df_analysis['year'] = df_analysis['date'].dt.year
df_analysis['month'] = df_analysis['date'].dt.month
df_analysis['day'] = df_analysis['date'].dt.day

[ ]
df_analysis.head()
```

	order_id	customer_id	sales_person	priority	ship_mode	date	sales_value	year	month	day
0	8999	C406	Kelly	High	Plane	2020-06-12	1553.44	2020	6	12
1	47872	C100	Kelly	Critical	Plane	2020-06-12	315.4	2020	6	12
2	29988	C006	Kelly	High	Lorry	2020-06-12	5075.89	2020	6	12
3	37348	C492	Kelly	High	Plane	2020-06-13	520.67	2020	6	13
4	28001	C231	Kelly	Critical	Plane	2020-06-14	8431.37	2020	6	14

Next steps: [Generate code with df_analysis](#) [New interactive sheet](#)

```
[ ] df_analysis.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001 entries, 0 to 1000
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   order_id        1001 non-null   Int64
1   customer_id     1001 non-null   string
2   sales_person    1001 non-null   string
3   priority        1001 non-null   string
4   ship_mode       1001 non-null   string
5   date            1001 non-null   datetime64[ns]
6   sales_value     1001 non-null   Float64
7   year            1001 non-null   int32
8   month           1001 non-null   int32
9   day             1001 non-null   int32
dtypes: Float64(1), Int64(1), datetime64[ns](1), int32(3), string(4)
memory usage: 68.6 KB
```

```
[ ] df_analysis.describe()

...      order_id      date  sales_value  year  month  day
count      1001.0      1001      1001.0  1001.0  1001.000000  1001.000000
mean  30231.488511  2020-09-04 08:03:21.398601216  1778.402248  2020.0  8.635365  15.876124
min         224.0      2020-06-12 00:00:00          2.45  2020.0  6.000000  1.000000
25%        16451.0      2020-07-26 00:00:00         134.4  2020.0  7.000000  8.000000
50%        29667.0      2020-09-04 00:00:00         417.58  2020.0  9.000000  16.000000
75%        44583.0      2020-10-15 00:00:00        1816.93  2020.0  10.000000  23.000000
max        59591.0      2020-11-30 00:00:00       27448.24  2020.0  11.000000  31.000000
std        16744.67454          NaN  3457.457206      0.0  1.598095  8.854188
```

Data Analysis

```
Data Analysis

[59] ✓ 0s df_analysis['sales_value'].describe()

...      sales_value
count      1001.0
mean    1778.402248
std    3457.457206
min         2.45
25%        134.4
50%        417.58
75%       1816.93
max       27448.24

dtype: Float64
```

```
[60]
✓ Os
total_sales = df_analysis['sales_value'].sum()
average_sales = df_analysis['sales_value'].mean()
minimum_sales = df_analysis['sales_value'].min()
maximum_sales = df_analysis['sales_value'].max()

print("Total Sales:", total_sales)
print("Average Sales:", average_sales)
print("Minimum Sales:", minimum_sales)
print("Maximum Sales:", maximum_sales)

Total Sales: 1780180.65
Average Sales: 1778.4022477522476
Minimum Sales: 2.45
Maximum Sales: 27448.24

[61]
✓ Os
total_orders = df_analysis['order_id'].count()
total_orders

... np.int64(1001)
```

```
[62]
✓ Os
# Total sales by salesperson
salesperson_total = df_analysis.groupby('sales_person')['sales_value'].sum()

salesperson_total

sales_value
sales_person
Kelly      558659.62
Peter      549292.06
Pryia      672228.97

dtype: Float64
```

```
[63]
✓ Os
salesperson_average = df_analysis.groupby('sales_person')['sales_value'].mean()

salesperson_average

sales_value
sales_person
Kelly      1751.284075
Peter      1596.779244
Pryia      1988.843107

dtype: Float64
```

```
[64]
✓ Os
salesperson_orders = df_analysis.groupby('sales_person')['order_id'].count()

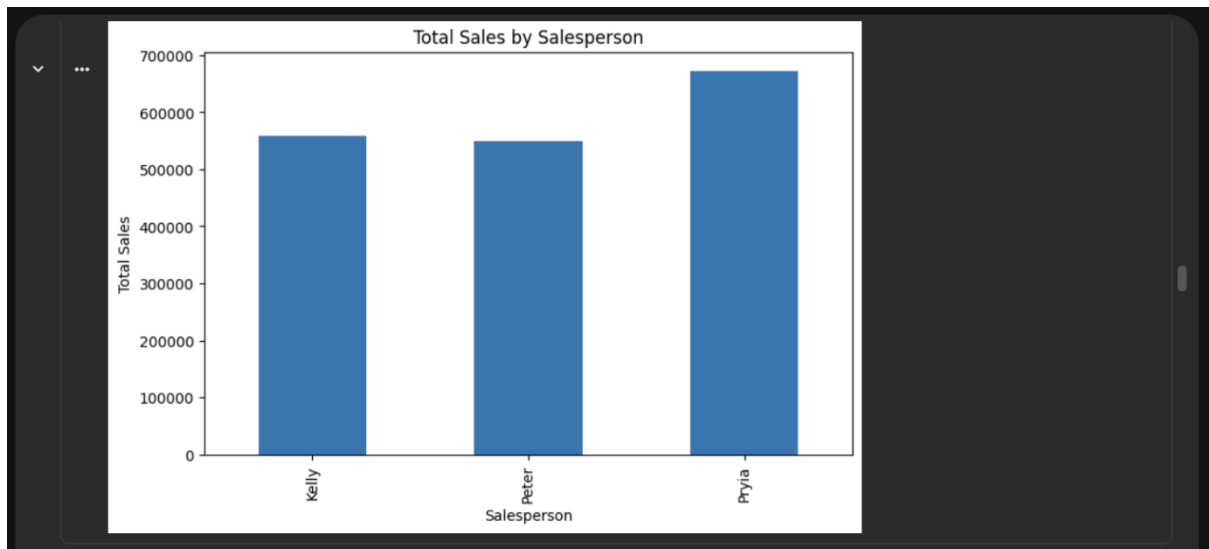
salesperson_orders

order_id
sales_person
Kelly      319
Peter      344
Pryia      338

dtype: Int64
```

```
[65]
✓ Os
salesperson_total.plot(
    kind='bar',
    figsize=(8,5)
)

plt.title("Total Sales by Salesperson")
plt.xlabel("Salesperson")
plt.ylabel("Total Sales")
plt.show()
```



```
[66] ✓ 0s
# Total sales by priority
priority_total = df_analysis.groupby('priority')['sales_value'].sum()
priority_total
```

priority	sales_value
Critical	403652.33
High	381543.4
Low	292535.81
Medium	330337.37
Not Specified	372111.74

dtype: Float64

```
[67] ✓ 0s
priority_average = df_analysis.groupby('priority')['sales_value'].mean()
priority_average
```

priority	sales_value
Critical	1826.481131
High	1946.65
Low	1462.67905
Medium	1931.797485
Not Specified	1747.003474

dtype: Float64

```
ProgrammingDAAssignment.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
```

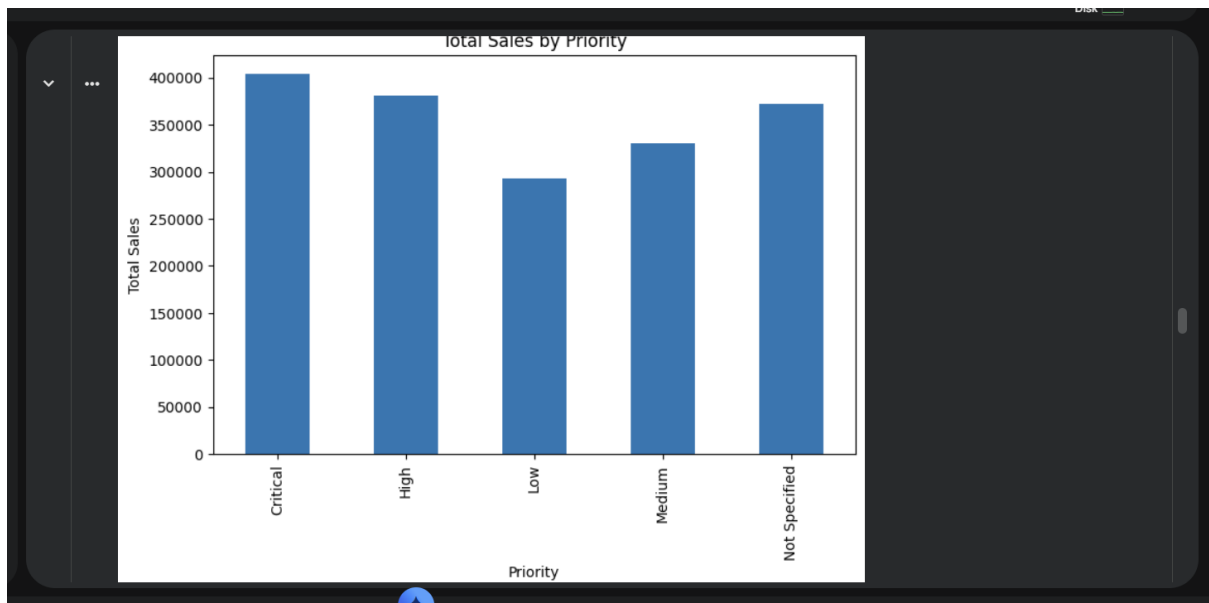
priority	order_id
Critical	221
High	196
Low	200
Medium	171
Not Specified	213

dtype: Int64

```
[69]
✓ 0s

priority_total.plot(
    kind='bar',
    figsize=(8,5)
)

plt.title("Total Sales by Priority")
plt.xlabel("Priority")
plt.ylabel("Total Sales")
plt.show()
```



```
[70]
✓ 0s

priority_average = df_analysis.groupby('priority')['sales_value'].mean()
priority_average
```

	sales_value
priority	
Critical	1826.481131
High	1946.65
Low	1462.67905
Medium	1931.797485
Not Specified	1747.003474

dtype: Float64

```
[71]
✓ 0s

df_analysis.boxplot(
    column='sales_value',
    by='priority',
    figsize=(10,6)
)

plt.title("Sales Value Distribution by Priority")
plt.suptitle("")
plt.xlabel("Priority")
plt.ylabel("Sales Value")
plt.show()
```



```
[72] ship_total = df_analysis.groupby('ship_mode')['sales_value'].sum()
✓ 0s ship_total
```

sales_value

ship_mode	sales_value
Lorry	627825.97
Plane	1018909.07
Van	133445.61

dtype: Float64

☆ Help

Share

RAM
Disk

```
[73] ship_average = df_analysis.groupby('ship_mode')['sales_value'].mean()
✓ 0s ship_average
```

sales_value

ship_mode	sales_value
Lorry	4982.745794
Plane	1323.258532
Van	1270.910571

dtype: Float64

```
[74]
✓ 0s
ship_orders = df_analysis.groupby('ship_mode')['order_id'].count()
ship_orders
```

ship_mode	order_id
Lorry	126
Plane	770
Van	105

dtype: Int64

```
[75]
✓ 0s
ship_total.plot(
    kind='bar',
    figsize=(8,5)
)

plt.title("Total Sales by Shipping Mode")
plt.xlabel("Shipping Mode")
plt.ylabel("Total Sales")
plt.show()
```



```
[76]
✓ 0s
monthly_sales = df_analysis.groupby('month')['sales_value'].sum()
monthly_sales
```

month	sales_value
6	144279.72
7	378143.61
8	332645.49
9	324590.99
10	355178.46
11	245342.38

dtype: Float64


```
[77] monthly_orders = df_analysis.groupby('month')['order_id'].count()
✓ 0s monthly_orders
```

month	order_id
6	99
7	190
8	193
9	183
10	167
11	169

dtype: Int64

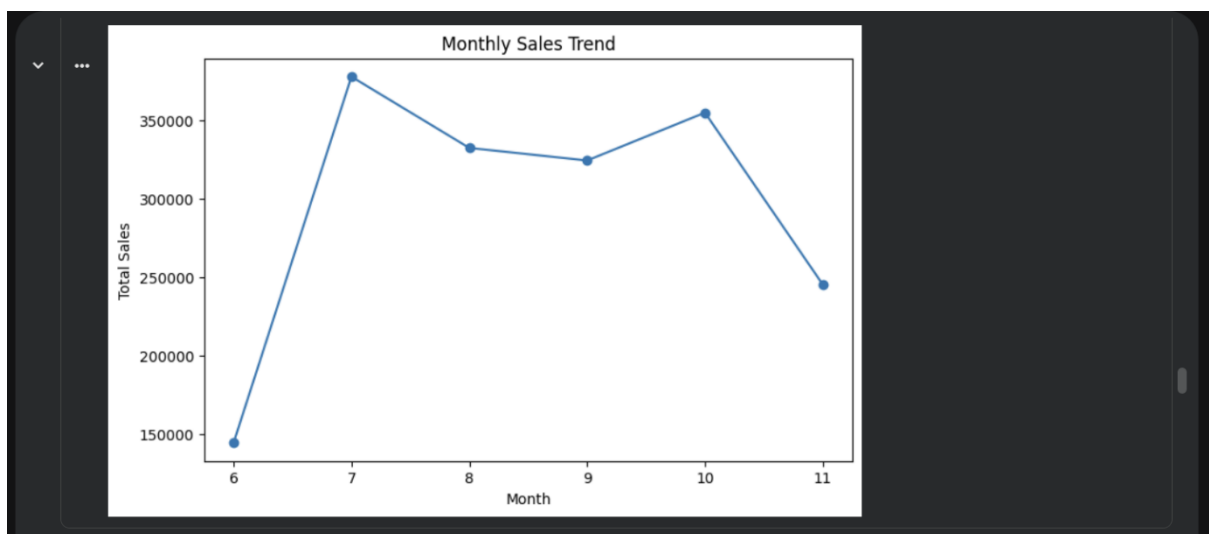
```
[78] monthly_average = df_analysis.groupby('month')['sales_value'].mean()
✓ 0s monthly_average
```

month	sales_value
6	1457.370909
7	1990.229526
8	1723.551762
9	1773.721257
10	2126.817126
11	1451.730059

dtype: Float64

```
[79] monthly_sales.plot(
✓ 0s     kind='line',
     marker='o',
     figsize=(8,5)
)

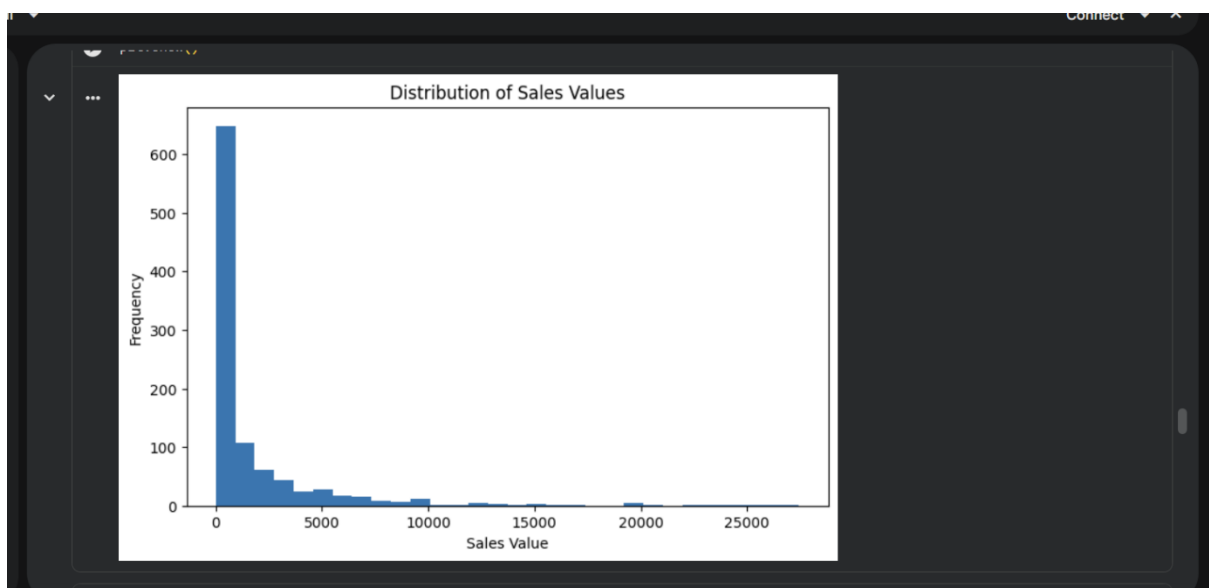
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.show()
```



Data Visualisation

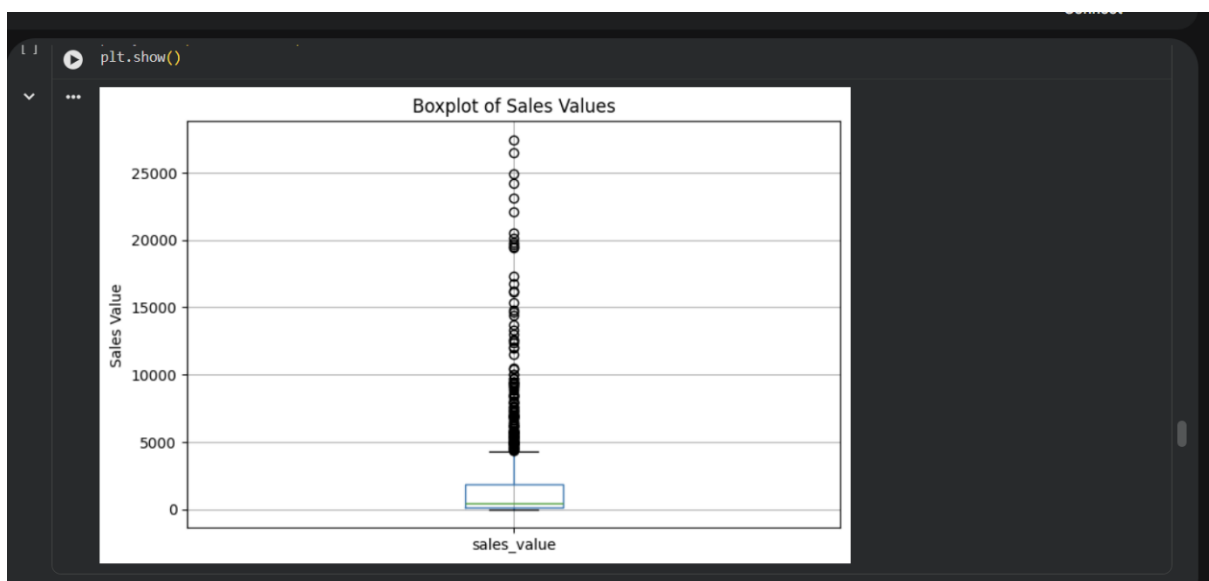
```
ProgrammingDAAssignment.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Table of contents
+ Section
Data Visualisation
[]
# Histogram of sales values
df_analysis['sales_value'].plot(
    kind='hist',
    bins=30,
    figsize=(8,5)
)

plt.title("Distribution of Sales Values")
plt.xlabel("Sales Value")
plt.ylabel("Frequency")
plt.show()
```



```
df_analysis.boxplot(
    column='sales_value',
    figsize=(8,5)
)

plt.title("Boxplot of Sales Values")
plt.ylabel("Sales Value")
plt.show()
```

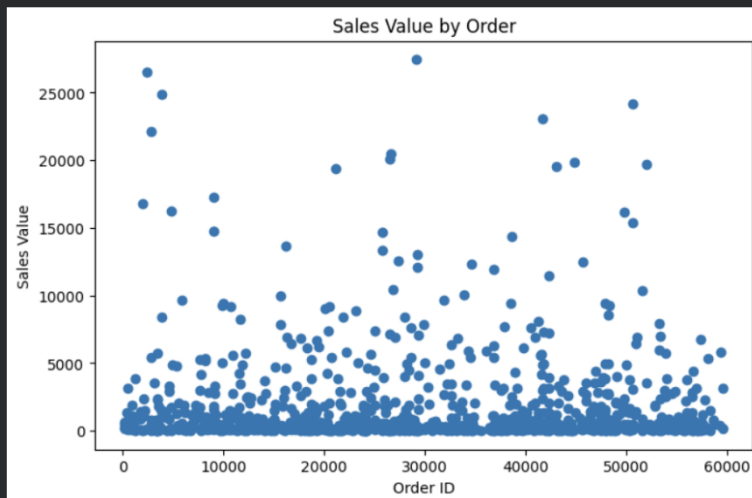


```
[ ] ▶ plt.figure(figsize=(8,5))

plt.scatter(
    df_analysis['order_id'],
    df_analysis['sales_value']
)

plt.title("Sales Value by Order")
plt.xlabel("Order ID")
plt.ylabel("Sales Value")

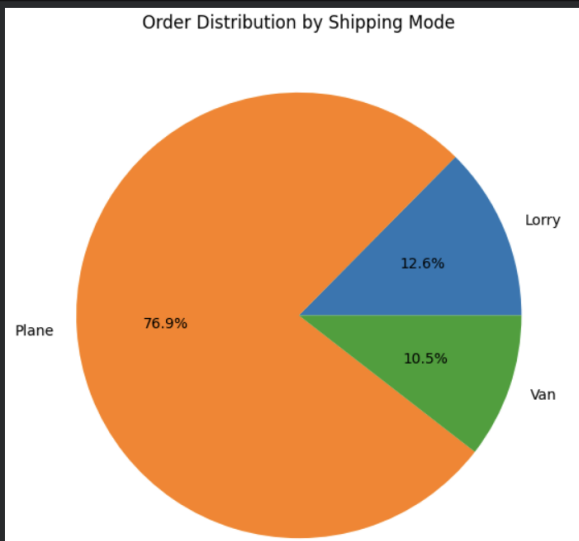
plt.show()
```



```
[ ] ▶ ship_orders.plot(
    kind='pie',
    autopct='%1.1f%%',
    figsize=(7,7)
)

plt.title("Order Distribution by Shipping Mode")
plt.ylabel("")

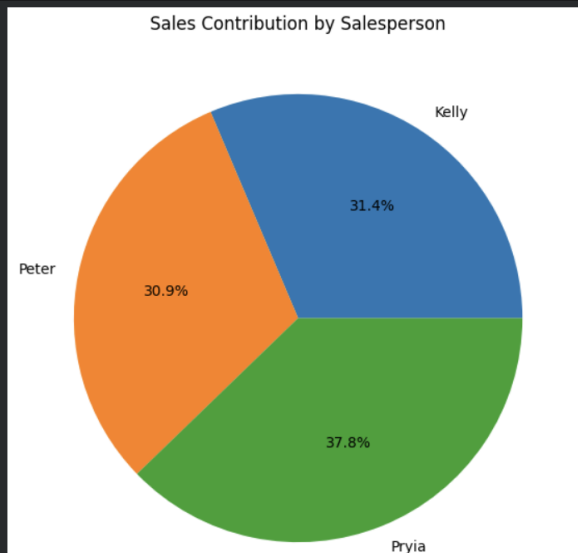
plt.show()
```



```
[1] salesperson_total.plot(
    kind='pie',
    autopct='%1.1f%%',
    figsize=(7,7)
)

plt.title("Sales Contribution by Salesperson")
plt.ylabel("")

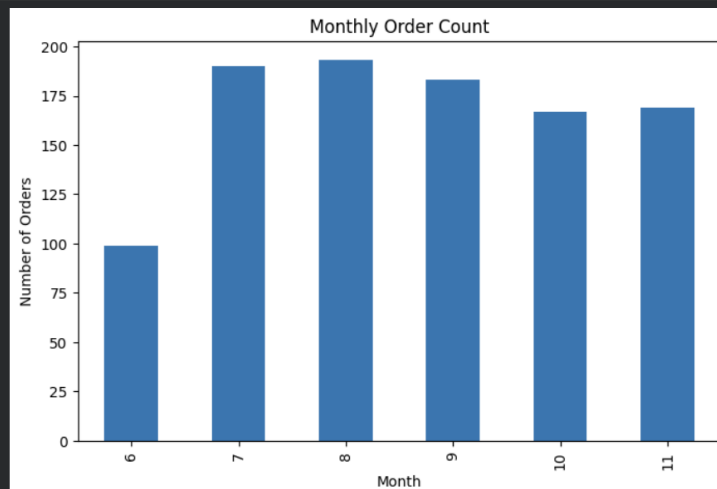
plt.show()
```



```
[1] monthly_orders.plot(
    kind='bar',
    figsize=(8,5)
)

plt.title("Monthly Order Count")
plt.xlabel("Month")
plt.ylabel("Number of Orders")

plt.show()
```



```
[ ] ▶ import seaborn as sns

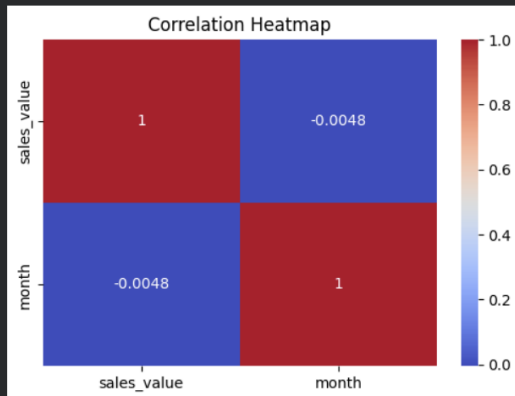
correlation_data = df_analysis[['sales_value', 'month']].corr()

plt.figure(figsize=(6,4))

sns.heatmap(
    correlation_data,
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")

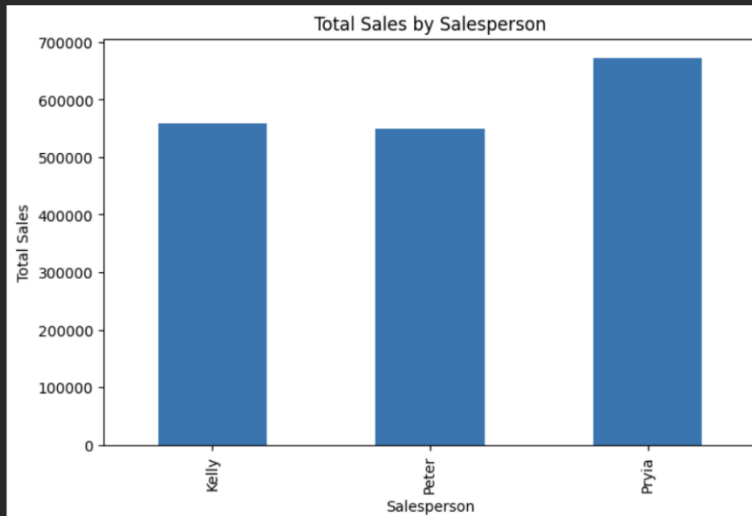
plt.show()
```



```
[ ] ▶ salesperson_total = df_analysis.groupby('sales_person')['sales_value'].sum()

salesperson_total.plot(
    kind='bar',
    figsize=(8,5)
)

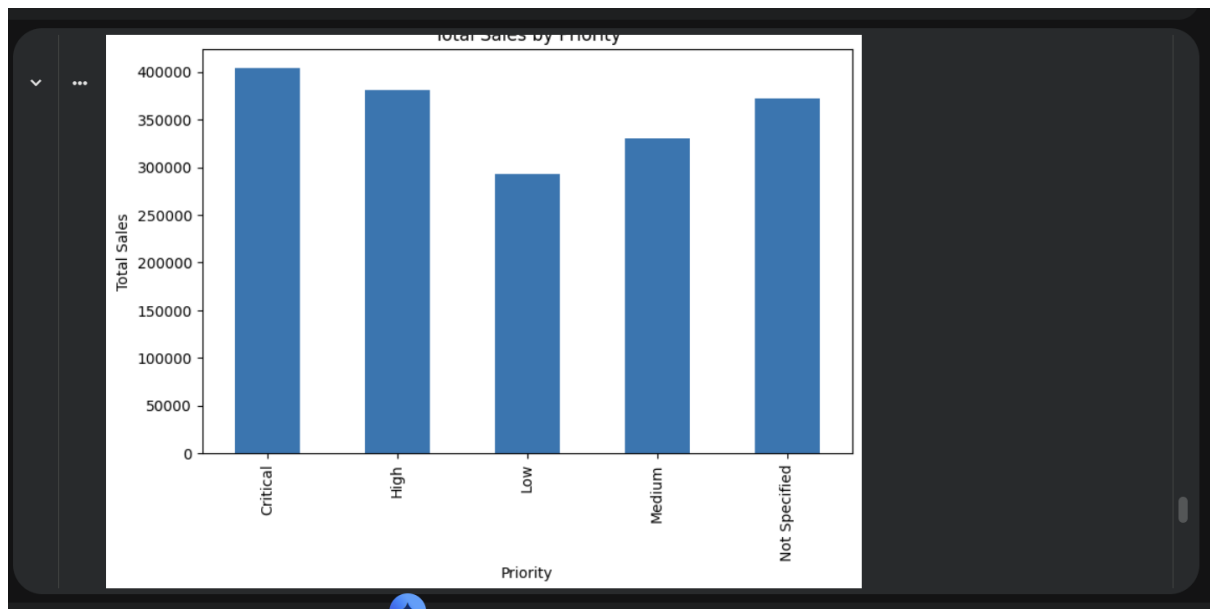
plt.title("Total Sales by Salesperson")
plt.xlabel("Salesperson")
plt.ylabel("Total Sales")
plt.show()
```



```
[ ] ▶ priority_total = df_analysis.groupby('priority')['sales_value'].sum()

priority_total.plot(
    kind='bar',
    figsize=(8,5)
)

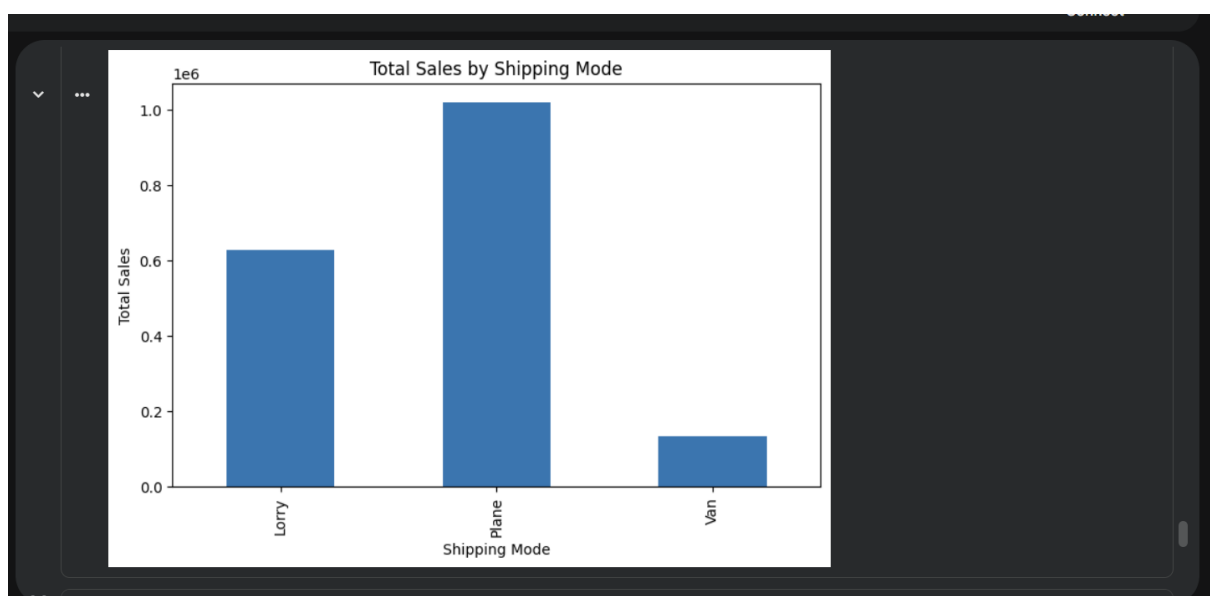
plt.title("Total Sales by Priority")
plt.xlabel("Priority")
plt.ylabel("Total Sales")
plt.show()
```



```
[ ] ▶ ship_total = df_analysis.groupby('ship_mode')['sales_value'].sum()

ship_total.plot(
    kind='bar',
    figsize=(8,5)
)

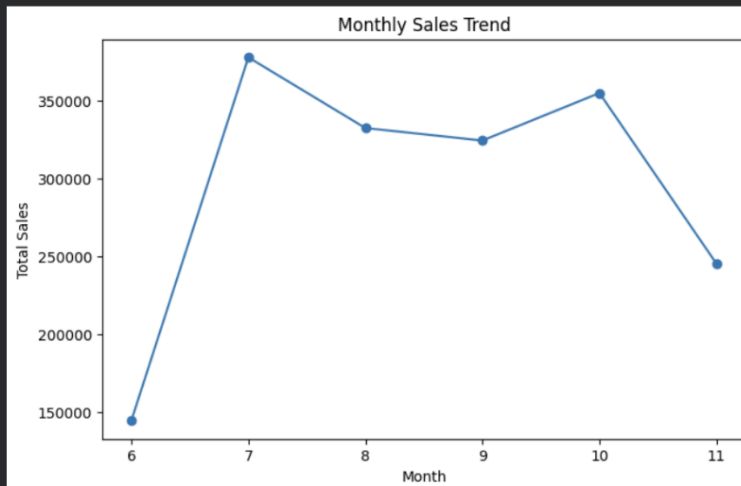
plt.title("Total Sales by Shipping Mode")
plt.xlabel("Shipping Mode")
plt.ylabel("Total Sales")
plt.show()
```



```
[ ] ▶ monthly_sales = df_analysis.groupby('month')['sales_value'].sum()

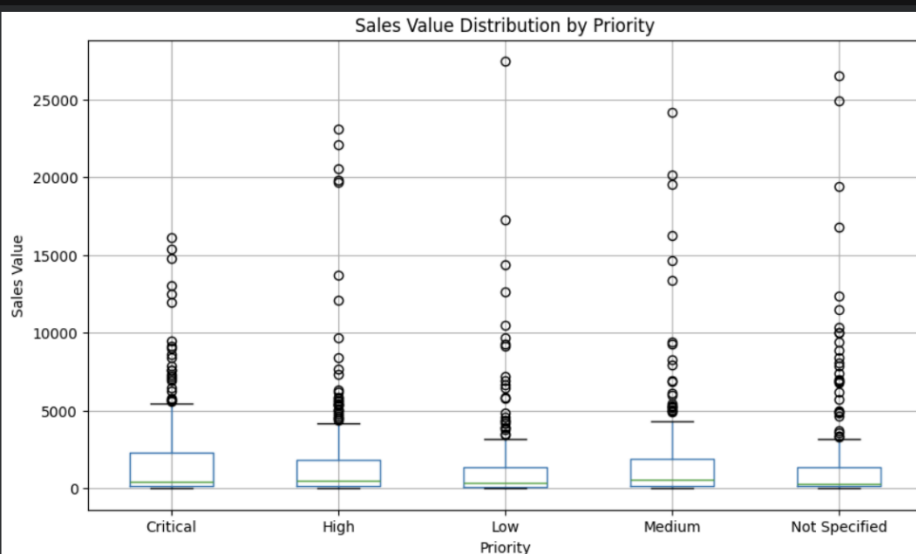
monthly_sales.plot(
    kind='line',
    marker='o',
    figsize=(8,5)
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.show()
```



```
[ ] ▶ df_analysis.boxplot(
    column='sales_value',
    by='priority',
    figsize=(10,6)
)

plt.title("Sales Value Distribution by Priority")
plt.suptitle("")
plt.xlabel("Priority")
plt.ylabel("Sales Value")
plt.show()
```



```
[1]
priority_mapping = {
    'Low': 1,
    'Medium': 2,
    'High': 3,
    'Critical': 4,
    'Not Specified': 0
}

df_analysis['priority_numeric'] = df_analysis['priority'].map(priority_mapping)

corr_data = df_analysis[['sales_value', 'priority_numeric']]

correlation_matrix = corr_data.corr()

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm'
```

```
[1]

corr_data = df_analysis[['sales_value', 'priority_numeric']]

correlation_matrix = corr_data.corr()

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap: Priority vs Sales Value")

plt.show()
```

